ML Problem Patterns from the Prateek Interview

Oct 14, 2026 · @Shahriar

The interview with Prateek Singh (a large GCC retail group) raised 11 real ML and AI challenges. Each one is turned here into a general problem that shows up in almost every industry.

Every problem card has the same parts:

- **From the interview:** what was actually said.
- **General problem:** the same challenge with the retail details removed.
- **Why it is hard:** the real difficulty, not the obvious one.
- **How to solve it:** a practical approach, step by step.
- **How to measure it:** what proves it works.
- **Common mistakes:** what usually goes wrong.
- **Same problem elsewhere:** how it looks in other industries.

The cards also work as interview practice and as test scenarios for DCLab, since each one has a clear right and wrong way to solve it.

The 11 challenges at a glance

Four of the 11 come from the fraud case. The rest come from deployment, the AI product work and how the team runs.

#	What came up in the interview	General problem pattern	Main ML task
1	Staff file fake returns on copied bills; millions of POS transactions, no labels	Anomaly and fraud detection with no labels	Unsupervised anomaly detection
2	Started with 0 labels, now 70,000 using rules, the Loss Prevention team and clustering	Building a labelled dataset from nothing	Weak supervision, active learning

#	What came up in the interview	General problem pattern	Main ML task
3	Catch every fraud first, then cut false alarms; use LLMs in the middle	Two-stage screening: high recall, then high precision	Cascade design, threshold tuning
4	A copy of a real bill is used for a fake return	Finding near-duplicates and linked records	Similarity learning, record matching
5	"How well do you own deployment?" Precision drops sharply on new data	Running models in production over time	MLOps, drift detection
6	Retail IQ: "Why are Aldo sales down in Dubai Mall?"	Ask-your-data BI and root-cause analysis	Text-to-SQL, metric decomposition
7	Sims GPT on Amazon Bedrock; data stays inside the company	A secure internal LLM platform	LLM gateway, governance
8	Finance, merchandising and pricing agents	Department agents that act on company data	Agents with tools and guardrails
9	Some problems need only OCR and Python, others need LLMs	Choosing the simplest tool that works	Problem triage
10	"Apparel Brain": collect all data in one place within 12 to 18 months	One shared data foundation for the whole company	Data platform, entity resolution
11	5 to 6 projects in 6 months, 15 in the pipeline	Delivering many AI projects fast	Prioritization, reusable components

Problem 1: Anomaly and fraud detection with no labels

From the interview: About 2,500 to 3,000 stores create millions of POS transactions. Staff copy a real bill and, after 11 p.m., file a fake return to take the money. Rules suggest about 10,000 of every 100,000 bills may be fake, but nobody knows which.

General problem: You have huge volumes of events, a few bad ones hidden inside, and no list of which are bad.

Why it is hard:

- Without labels you cannot compute precision or recall, so you cannot tell if a model works.
- "Unusual" is not the same as "fraud". A big holiday sale is unusual but honest.
- Fraudsters copy normal behaviour, so the strongest signal is often in relations (same bill, same cashier, same hour), not in one row.
- The fraud rate is low and changes when people learn they are being watched.

How to solve it:

1. Profile the data: missing values, time patterns, per-store and per-cashier baselines.
2. Build features at the right level: per transaction, per cashier per shift, per store per day. Most fraud shows up in aggregates.
3. Score with several unsupervised views: Isolation Forest for global outliers, an autoencoder for reconstruction error, rules as features, and peer comparison ("this cashier vs others in the same store").
4. Rank by risk and send the top cases to human reviewers. Their answers become the first labels (Problem 2).
5. Once labels exist, move to a supervised model and keep the unsupervised score as one feature.

How to measure it:

- Precision@K: of the top 100 or 1,000 flagged cases, how many reviewers confirm.
- Recall on a small random audit sample, which is the only honest way to estimate missed fraud.
- Money recovered per reviewer hour.

Common mistakes:

- Reporting accuracy. With 1% fraud, a model that flags nothing is 99% accurate.
- Running DBSCAN on millions of rows. It does not scale; use a sample or HDBSCAN.
- Splitting train and test randomly. Split by time, and keep stores or cashiers apart when you can.

Same problem elsewhere: card and payment fraud (banking), fake insurance claims, SIM-box fraud (telecom), billing fraud (healthcare), fake reviews and coupon abuse (e-commerce), fuel-card misuse (logistics), and abnormal machine readings (manufacturing).

Problem 2: Building a labelled dataset from nothing

From the interview: The team began with 10 to 20 business rules and zero labels. The Loss Prevention team confirmed real fraud cases, and k-means clustering found similar cases. Labels grew from 0 to about 20,000, then to about 70,000. Prateek's view: good data solves 70 to 80% of the problem.

General problem: You need a supervised model, but labels are expensive because only experts can give them.

Why it is hard:

- Rule-based labels copy the rules' blind spots. A model trained on them learns the rules, not the fraud.
- Experts only review what you show them, so cases no rule flags never get a label.
- Clustering spreads labels to neighbours, which also spreads mistakes.

How to solve it:

1. Treat rules as noisy votes, not truth. Weak-supervision tools such as Snorkel combine many rules into one probability per case.
2. Use active learning: send experts the cases the model is least sure about, not only the highest scores.
3. Always add a small random sample to every review batch. It is the only way to find fraud the rules miss.
4. Record who labelled each case, when, and from which source (rule, expert, cluster). Weight sources by how reliable they prove to be.
5. Measure agreement between two reviewers on the same cases. Low agreement means the label definition is unclear.

How to measure it:

- Model quality per 1,000 new labels (the learning curve).
- Precision of each label source against expert-confirmed cases.
- Share of confirmed fraud that no rule would have flagged.

Common mistakes:

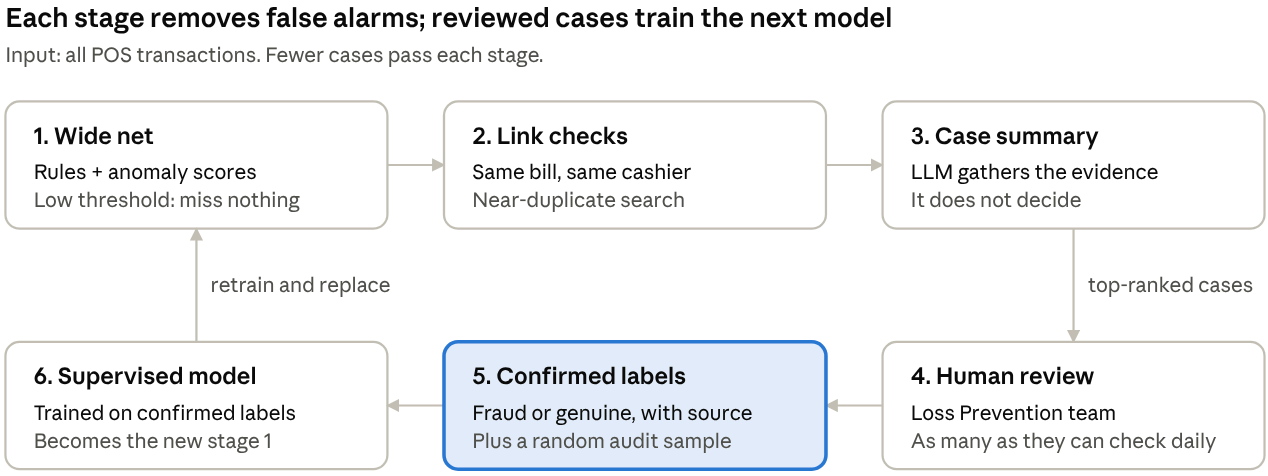
- Evaluating on labels made by the same rules the model was trained on. The score looks great and means nothing.
- Mixing guessed labels (from clustering) and confirmed labels without marking which is which.

Same problem elsewhere: medical image labels that need doctors, legal document review, content moderation, money-laundering alerts that need investigators, and defect labels in factory inspection.

Problem 3: Two-stage screening, high recall first, then high precision

From the interview: Shahriar proposed aiming for high recall first (lower the threshold so no fraud is missed), then using LLMs and similarity models in the middle of the pipeline to filter cases and raise precision. Prateek said this was very close to what they do, and that labelling is the main bottleneck.

General problem: Positive cases are rare and every check by a person is expensive. You need many cheap filters first and few expensive checks last.



fraud screening cascade · 6 stages, one feedback loop

The loop at the bottom is the key: every reviewed case becomes a label, and the labels replace the rules with a trained model.

Why it is hard:

- High recall floods reviewers with false alarms, and the team can check only so many cases a day.
- A fraud case dropped at stage 1 is never seen again, so final precision hides early misses.

How to solve it:

1. Set the stage 1 threshold from a recall target on known fraud, then check that the volume fits review capacity.
2. Put cheap, exact checks next (Problem 4) before anything expensive.
3. Use the LLM to read notes and receipts and write a short case summary for reviewers. The decision stays with people and models.
4. If reviewers can check K cases a day, the review threshold is simply the score of the K-th case.
5. Calibrate scores so 0.8 really means about 80%, then rank by probability × money at risk, not by probability alone.

How to measure it: recall of each stage on confirmed fraud, precision@K at review, review minutes per case, and money recovered per day.

Common mistakes: measuring only the last stage; letting the LLM label cases as fraud by itself.

Same problem elsewhere: money-laundering alert queues, cancer screening (cheap test, then biopsy), CV screening, content moderation, and search (fast retrieval, then a slower re-ranker).

Problem 4: Finding near-duplicates and linked records

From the interview: The fraud works by copying a real bill and using the copy for a fake return. Shahriar suggested Siamese networks, which learn whether two items are the same or different.

General problem: Two records that look different are really the same thing, or are linked in a suspicious way.

Why it is hard:

- Comparing every pair is impossible at scale: 1 million bills means about 500 billion pairs.
- Copies are rarely exact. Amounts, times or IDs change slightly.
- Many matches are honest (a real customer returning a real item).

How to solve it:

1. Start with exact links that cost nothing: same bill ID returned twice, a return linked to a bill from another store, a return processed by the cashier who made the sale.
2. Use blocking to cut the pairs: only compare records that share a store, a day or an amount range.

3. For fuzzy matches, turn each record into a vector (an embedding) and use nearest-neighbour search (for example FAISS).
4. If bills are scanned images, a Siamese or contrastive model can compare the images themselves.
5. Build a graph of cashier, customer, bill and return. Rings of linked cases are often clearer than single cases.

How to measure it: match precision and recall on a hand-checked set of pairs, and the number of fraud rings found that single-case scoring missed.

Common mistakes: jumping to a deep Siamese model when a SQL join on bill ID and cashier would catch most cases.

Same problem elsewhere: duplicate insurance claims, the same patient under two IDs, fake accounts by one person (multi-accounting), duplicate supplier invoices, near-duplicate images leaking between train and test sets, and duplicate listings in marketplaces.

Problem 5: Running models in production over time

From the interview: Prateek asked how well Shahriar owns deployment. Shahriar described building his own API, deploying on Cloud Run, SageMaker or ECS, monitoring with MLflow or Weights & Biases, blue-green releases, and precision dropping sharply on new data because of drift or hidden overfitting.

General problem: A model that scored well in testing gets worse in the real world, and often nobody notices for weeks.

Why it is hard:

- True labels arrive late. A fraud case may be confirmed a month after the transaction.
- Three different causes look the same from outside: data drift (inputs change), concept drift (what counts as fraud changes) and leakage or overfitting (the test score was never real).
- In fraud, the people being caught change their behaviour on purpose.

How to solve it:

1. Before launch, test on a later time period than training. If the score falls there, it will fall in production too.

2. Monitor what you can see at once: input distributions (PSI), share of cases flagged, score distribution.
3. Monitor what arrives late: precision on reviewed cases, recall on audit samples.
4. Release safely: shadow mode first (the new model scores but does not act), then a canary or blue-green switch with one-click rollback.
5. Retrain on a schedule and when drift alerts fire, and check the new version against the old one on the same recent data.

How to measure it: time from drift to detection, time to roll back, and the gap between offline and live precision.

Common mistakes: watching only server metrics (latency, errors) and not model quality; retraining on the latest data without checking that the new model is actually better.

Same problem elsewhere: credit scoring after an interest-rate change, demand forecasts after a pandemic or a new competitor, spam filters, recommendation models after a catalogue change, and churn models after a new price plan.

Problem 6: Ask-your-data BI and root-cause analysis

From the interview: Retail IQ replaces fixed Power BI and MicroStrategy dashboards. A brand head asks "Why are Aldo sales down in Dubai Mall vs last year?" An LLM selects (not writes) the right SQL, gets the data and explains it: footfall fell from about 1 million to about 800,000. The next step is action, such as personalized offers.

General problem: Business users want answers to new questions in minutes, without waiting for an analyst, and they want the reason, not just the number.

Why it is hard:

- A wrong number that sounds confident is worse than no answer. Free text-to-SQL often joins the wrong tables or uses the wrong date range.
- "Why" needs a breakdown of the change, not one query.
- Users ask about things the data cannot answer (for example competitor activity).

How to solve it:

1. Build a semantic layer first: approved metric definitions (sales, footfall, conversion) and approved joins.
2. Let the LLM pick from tested query templates and fill in parameters, as Retail IQ does. Allow free SQL only behind validation.

3. For "why" questions, decompose the change: sales = footfall × conversion rate × average basket. Show which factor moved and by how much.
4. Show the query, the data source and the date range under every answer so users can check it.
5. End with a suggested action and, later, measure whether the action helped.

How to measure it: accuracy on a test set of real business questions with known answers, share of questions answered without an analyst, and time to answer.

Common mistakes: stopping at "footfall dropped" without checking conversion and basket size; treating a correlation as the cause.

Same problem elsewhere: branch performance in banking, network KPIs in telecom, bed occupancy in hospitals, delivery delays in logistics, booking drops in travel, and campaign results in marketing.

Problem 7: A secure internal LLM platform

From the interview: Sims GPT is the company's own ChatGPT-style tool built on Amazon Bedrock. Staff can chat, upload files and run analyses, with access to several models including Claude. All inference runs inside the company's AWS setup, so data stays with them.

General problem: Employees want strong AI models, but the company cannot send customer or financial data to public tools.

Why it is hard:

- Security is more than hosting: who can see which files, and can one user's upload leak into another's answer?
- Uploaded files bring problems: large spreadsheets, scanned PDFs, and hidden instructions inside documents (prompt injection).
- Costs grow fast once thousands of staff use it, and nobody knows which use is worth it.

How to solve it:

1. Put one gateway in front of all models: login, per-team access rules, logging, cost per team, and model choice.
2. Route by task: a small, cheap model for simple questions, a frontier model for hard ones.

3. For file analysis, run real code on the data (a sandbox) instead of asking the model to guess numbers from text.
4. Mask personal data before it reaches the model, and keep audit logs of what was asked and answered.
5. Track usage per department so the next AI projects go where people already find value.

How to measure it: weekly active users, cost per useful answer, number of security incidents (target zero), and user-rated answer quality.

Common mistakes: treating "hosted in our cloud" as the whole security story; letting the model compute numbers in its head instead of in code.

Same problem elsewhere: banks and insurers with strict regulators, hospitals with patient records, law firms, government agencies, and any company under data-residency laws such as the UAE's.

Problem 8: Department agents that act on company data

From the interview: A finance agent, a merchandising agent and a pricing agent are in the pipeline. The goal is to put AI into every department.

General problem: Each department wants an assistant that does not only answer questions but takes steps: pulls reports, proposes prices, drafts orders.

Why it is hard:

- An agent that changes a price or an order can lose real money in one mistake.
- Each department has its own data, rules and vocabulary, so one general agent does poorly.
- Long multi-step tasks fail quietly: step 7 uses a wrong number from step 2.

How to solve it:

1. Start each agent read-only: it answers and recommends, a person approves.
2. Give it narrow, typed tools ("get_sales(store, period)", "propose_price(sku, new_price)") instead of open database access.
3. Put hard limits in code, not in the prompt: maximum price change, budget caps, approval above a set amount.
4. Let the numbers come from models and rules (forecast, price elasticity); the LLM explains and organizes.

5. Log every step, so any action can be traced and undone.
6. Allow automatic action only for low-risk decisions that the agent has already got right many times.

How to measure it: share of recommendations accepted by staff, money impact versus a control group, error rate, and hours saved per week.

Common mistakes: letting the LLM decide prices directly; building ten agents at once instead of making one reliable first.

Same problem elsewhere: claims-handling agents (insurance), credit-review agents (banking), route-planning agents (logistics), revenue-management agents (hotels and airlines), and procurement agents in manufacturing.

Problem 9: Choosing the simplest tool that works

From the interview: Business teams bring many requests. Some are plain automation where OCR and Python are enough. Others need LLMs, either small language models or frontier models.

General problem: Every request arrives as "we need AI", but the right answer ranges from a script to a large model.

Why it is hard: a complex solution looks impressive in a demo but costs more, breaks more often and is harder to explain. A too-simple one fails on messy real inputs.

How to solve it: ask these questions in order and stop at the first "yes".

Question	If yes, use	Example
Is the input fixed and the rule clear?	A script or rules	Moving data between two systems
Is the text printed in a fixed layout?	OCR + templates	Standard supplier invoices
Is it a prediction from table data with labels?	Classical ML (GBDT, logistic regression)	Churn, demand, fraud score
Is it a narrow language task with many examples?	A small or fine-tuned model	Ticket routing, field extraction
Does it need reasoning over messy, varied input?	A frontier LLM	Contract review, open questions

How to measure it: cost per task, accuracy on a fixed test set, and time to build. Compare the chosen tool with the next simpler one.

Common mistakes: using an LLM to do arithmetic or exact lookups; never testing whether a simpler method was already good enough.

Same problem elsewhere: every company with an AI team and a queue of requests. This triage is the same in banks, hospitals, airlines and government.

Problem 10: One shared data foundation for the whole company

From the interview: The long-term vision is an "Apparel Brain". Through a platform transcribed as "MCPT", the team is collecting all company data, with a target of 12 to 18 months.

General problem: A group with many brands, stores and systems wants one trusted place where every AI project can find clean, linked data.

Why it is hard:

- The same customer, product or store has different IDs in different systems.
- Each brand defines metrics differently (what counts as "a sale" after a return?).
- Big platform projects often take years before any user sees value.

How to solve it:

1. Build it through real use cases: each project adds the tables it needs, cleaned and documented, so value arrives every few months.
2. Agree on shared IDs and metric definitions early, and record them in a catalog.
3. Resolve entities: link the same customer or product across systems with rules plus matching models (Problem 4 again).
4. Keep history: store data as it was at each point in time, so models can be trained without future information.
5. Add data-quality checks and an owner for every important table.

How to measure it: time for a new AI project to get usable data, share of key tables with an owner and quality checks, and number of projects reusing the same tables.

Common mistakes: collecting everything first and planning use later; overwriting old values, which makes honest backtesting impossible.

Same problem elsewhere: bank groups with many products, hospital networks, airline groups with loyalty programs, telecom operators across countries, and conglomerates after acquisitions.

Problem 11: Delivering many AI projects fast

From the interview: The team deployed 5 or 6 projects in the last 6 months and has about 15 in the pipeline for the next 6. Expectations are high, and engineers also act as product managers who talk directly with the business.

General problem: A small AI team must roughly triple its output without lowering quality.

Why it is hard: every project is built from scratch, so each new one costs as much as the last. Projects that are shipped but not used still take maintenance time.

How to solve it:

1. Score every request on value, data readiness, effort and risk, and say no or "later" to the rest.
2. Reuse shared parts: one LLM gateway (Problem 7), one feature store, one deployment and monitoring setup (Problem 5), one evaluation harness.
3. Agree on a success number with the business owner before building, such as "reviewer hours per confirmed fraud case".
4. Ship a small first version within weeks and measure real usage.
5. Retire projects nobody uses, to free the team.

How to measure it: time from request to first live version, share of projects with a measured business result, and share of projects still in use after 3 months.

Common mistakes: counting projects shipped instead of value delivered; starting all 15 at once.

Same problem elsewhere: every central AI or data team in a large company. This is also the exact pain DCLab aims at: less time per ML project.

The same problems in other industries

The retail fraud case maps almost one to one onto banking, insurance and healthcare. The table gives one concrete use case per industry for the four patterns that travel best.

Industry	Fraud with no labels (P1, P2)	Near-duplicates (P4)	"Why did this number change?" (P6)	Department agent (P8)
Banking	Money-laundering patterns across accounts	One person behind many accounts	Why did card spend drop in one region?	Credit-review assistant
Insurance	Staged accidents, inflated claims	Same damage claimed twice with different photos	Why did the loss ratio rise this quarter?	Claims-triage agent
Telecom	SIM-box and subscription fraud	Re-registered customers who skip debt	Why did churn rise after the price change?	Retention-offer agent
Healthcare	Billing for services never given	One patient under two record IDs	Why did readmissions rise in one hospital?	Coding and billing assistant
E-commerce	Coupon abuse, return fraud, fake reviews	Duplicate listings, reseller accounts	Why did conversion fall on mobile?	Pricing and promotion agent
Logistics	Fuel-card misuse, fake delivery proof	Duplicate supplier invoices	Why are deliveries late in one city?	Route and dispatch agent

Industry	Fraud with no labels (P1, P2)	Near-duplicates (P4)	"Why did this number change?" (P6)	Department agent (P8)
Travel	Fake bookings, loyalty-point theft	Same traveller in many loyalty accounts	Why did bookings drop for one destination?	Revenue-management agent

Problems 5, 7, 9, 10 and 11 are not tied to an industry at all: every company with an AI team meets them.

Practice questions

Each question below is a variant of something from the interview. A strong answer names the metric, the validation split and the first simple baseline before any model.

1. An insurer has 2 million claims and no fraud labels. How do you find the first 500 cases worth reviewing, and how do you know your list is better than the current rules? (P1, P2)
2. You have 70,000 labels, but 60,000 came from rules. How do you train and test so the model learns fraud and not the rules? (P2)
3. Your fraud model must catch at least 95% of fraud, but the review team can check only 300 cases a day. How do you set the threshold? (P3)
4. A bank suspects one person runs 40 accounts. How do you find linked accounts among 5 million without comparing every pair? (P4)
5. Your model's precision fell from 0.80 to 0.55 in one month. Walk through how you decide between data drift, concept drift and leakage. (P5)
6. A manager asks the chatbot "Why did revenue fall 12% in Q3?" What must happen before the answer is shown? (P6)
7. A pricing agent suggests a 30% discount on 2,000 products. What stops it from being applied automatically? (P8)
8. Three teams each ask for "an AI solution" for invoices, emails and demand planning. Which tool fits each, and why? (P9)
9. Your company has the same customer in 4 systems with different IDs. How do you build one customer view without breaking existing reports? (P10)
10. You have 15 requested projects and capacity for 5. How do you choose, and what do you reuse? (P11)

For DCLab, problems 1 to 5 can become test scenarios with planted traps: a time-based leak (the 11 p.m. rule computed with future data), duplicate bills split across train and test, and a drift period in the holdout.

خلاصهٔ فارسی

مطرح شد، به ۱۱ مسئلهٔ عمومی تبدیل Prateek Singh این سند ۱۱ چالش واقعی را که در مصاحبه با می‌کند که تقریباً در همهٔ صنایع تکرار می‌شوند. هر مسئله شامل بخش‌های «از مصاحبه»، «مسئلهٔ عمومی»، «چرا سخت است»، «راه‌حل»، «سنجش»، «اشتباه‌های رایج» و «همین مسئله در صنایع دیگر» است.

شماره	مسئلهٔ عمومی	نمونه در صنایع دیگر
۱	تشخیص ناهنجاری و تقلب بدون برچسب	تقلب کارت بانکی، خسارت ساختگی بیمه
۲	ساختن دادهٔ برچسب‌دار از صفر	برچسب تصاویر پزشکی، هشدارهای پول‌شویی
۳	بالا، بعد Recall غربالگری دومرحله‌ای: اول بالا Precision	صف بررسی خسارت و هشدارهای بانکی
۴	یافتن رکوردهای تکراری و مرتبط	یک بیمار با دو شناسه، چند حساب برای یک نفر
۵	Drift نگهداری مدل در محیط واقعی و کنترل	امتیاز اعتباری پس از تغییر نرخ بهره
۶	پرسش از داده به زبان ساده و تحلیل علت	«چرا ریزش مشتری در مخابرات بالا رفت؟»
۷	LLM پلتفرم امن و داخلی	بانک‌ها، بیمارستان‌ها و دولت
۸	عامل‌های هوشمند برای هر بخش سازمان	عامل بررسی خسارت، عامل مدیریت درآمد هتل
۹	انتخاب ساده‌ترین ابزار کارآمد	هر تیم هوش مصنوعی با صف درخواست
۱۰	زیرساخت دادهٔ واحد برای کل سازمان	گروه‌های بانکی و شبکه‌های بیمارستانی
۱۱	تحویل سریع پروژه‌های زیاد	هر تیم مرکزی داده در شرکت‌های بزرگ

مهم‌ترین درس‌ها:

- را حساب کرد؛ بنابراین بررسی انسانی و نمونهٔ تصادفی Recall و Precision بدون برچسب نمی‌توان برای ممیزی از روز اول لازم است.
- قانون‌های کسب‌وکار باید به‌عنوان رأی نویزدار استفاده شوند، نه حقیقت قطعی؛ وگرنه مدل فقط همان قانون‌ها را یاد می‌گیرد.

- تقسیم داده‌های آموزش و ارزیابی نهایی (Holdout) باید بر اساس زمان باشد تا نشت داده رخ ندهد.
- LLM فقط توضیح می‌دهد و سازمان‌دهی اعداد و تصمیم‌های مالی را مدل‌ها و قانون‌ها تولید می‌کنند و می‌کند.
- عامل هوشمند ابتدا فقط پیشنهاد می‌دهد و تأیید انسانی لازم است؛ اجرای خودکار تنها برای تصمیم‌های کم‌ریسک و پس از اثبات دقت مجاز است.

بخش «پرسش‌های تمرینی» برای آمادگی مصاحبه و همچنین برای ساخت سناریوهای آزمون در DCLab قابل استفاده است.